

REVERSE ENGINEERING // MASTERY COURSE

PHASE 4

Dynamic Analysis

Chalate hue samajhna — breakpoints, live values, patching.

```
● ● ● re@learning: ~/phase-4

$ ./start_learning.sh --phase 4
# Hinglish mein, simple examples ke saath — ek-ek step
$ status: READY ✓
```

📌 Detailed Theory

📄 Code + Line-by-line

🎯 Interview Q&A

IN Hinglish

Reverse Engineering Mastery • Self-Study Roadmap 1 • 21 Phase 4 Handbook

Phase 4 — Dynamic Analysis (Chalate Hue Samajhna)

Goal: Program ko **chalate hue** rok-rok ke samajhna. Jab static se samajh na aaye (ya code chhupa ho), tab ye kaam aata hai. Time: **3-4 hafte**.

4.0 — Dynamic Analysis Kya Hai?

Dynamic analysis matlab program ko **actually chalakar**, beech mein rok-rok ke, live values dekhkar samajhna. Static "book padhna" tha, dynamic "movie dekhna" hai — cheezein hoti hui dikhti hain.

Kab use karein: - Code encrypted/packed hai (static mein samajh nahi aata). - Password runtime pe decrypt/generate hota hai. - Bahut complex logic — chala ke dekhna aasan hai.

 **Safety:** Malware ya unknown files hamesha **Virtual Machine (VM)** mein chalao, apne main PC pe kabhi nahi. (VM setup Phase 5 mein.)

4.1 — Debugger Ki Power

Debugger program ko control mein leta hai. Tum decide karte ho kab chale, kab ruke, aur har point pe sab kuch dekh sakte ho.

Basic Concepts

Breakpoint — ek nishaan jahan program apne aap ruk jaayega. Sabse important tool. - Software breakpoint (**F2** in x64dbg) — kisi instruction pe. - Conditional breakpoint — sirf jab koi condition sachh ho (jaise **eax == 5**). - Hardware breakpoint — memory read/write pe ruk jaana.

Stepping — ek-ek karke chalana: | Action | x64dbg | GDB | Kaam | |-----|-----|-----|-----| | Step Into | **F7** | **stepi** / **s** | function ke andar ghuso | | Step Over | **F8** | **nexti** / **n** | function ko chala do, andar mat jao | | Step Out | **Ctrl+F9** | **finish** | current function se bahar niklo | | Run/Continue | **F9** | **continue** / **c** | agle breakpoint tak chalao |

Watch/Inspect — chalate hue dekhna: - **Registers** — live values (eax, eip...). - **Memory/Dump** — kisi address pe kya rakha hai (hex + ASCII). - **Stack** — abhi stack pe kya hai.

4.2 — Runtime Values Pakadna (Sabse Bada Fayda)

Static mein bas code dikhta hai, dynamic mein **asli values** dikhti hain.

Example scenario: Ek program password ko encrypt karke compare karta hai. Static analysis mein tumhe sirf encrypted bytes dikhenge — asli password nahi. Lekin:

1. `strcmp` (ya jahan comparison hota hai) pe **breakpoint** lagao.
2. Program chalao (`F9`), password wahan pe rukega.
3. Registers/stack dekho — `strcmp` ke arguments mein **decrypt hua asli password** memory mein dikhega! 🕵️

Ye dynamic analysis ka superpower hai — encryption ke baad ki asli value seedha memory mein dekh lena, bina encryption samjhe.

Kahan breakpoint lagayein? (common spots)

- `strcmp` / `strncmp` / `memcmp` — comparisons pe.
- `scanf` / `fgets` ke baad — input padhne ke baad.
- Interesting function ke shuru mein.
- Jahan "success/wrong" message print hone se pehle decision hota hai.

4.3 — Patching (Code Badalna)

Patching matlab program ke bytes/instructions ko badal ke uska behaviour change karna. Ye RE ka bahut powerful (aur maze wala) hissa hai.

Classic Example: "Wrong" ko "Right" bana dena

Password check aisa hota hai:

```
cmp eax, ebx      ; password sahi hai?
jne wrong_password ; agar NAHI (not equal), wrong pe jao
; ... access granted ...
wrong_password:
; ... wrong message ...
```

Patch idea: `jne` (jump if not equal) ko `je` (jump if equal) ya `nop` (kuch mat karo) mein badal do. Ab galat password pe bhi "access granted" chalega!

```
cmp eax, ebx
je wrong_password ; ULTA kar diya (ya nop se hata do)
```

Patching ke tarike

- **NOP karna** — instruction ko `nop` (No Operation, `0x90`) se replace karna — matlab "kuch mat karo, aage badho".
- **Jump ulta karna** — `je` ↔ `jne`, `jg` ↔ `jle` etc.
- **Return value badalna** — function ko forcefully `return 1` (ya 0) karwana.

x64dbg mein patch kaise

1. Jis instruction ko badalna hai uspe right-click → "Assemble" (`Space`).
2. Nayi instruction likho (jaise `nop` ya `je`).
3. File → "Patch file" → save karke naya `.exe` banao.

⚠ Patching sirf learning ya authorized testing ke liye. Kisi ki software ki protection todna illegal hai.

4.4 — Static + Dynamic Combo (Real Workflow)

Asli RE mein dono milaake use hote hain:

1. Static (Ghidra) → overall structure samjho, interesting functions dhoondo
2. Breakpoints → un functions pe x64dbg mein breakpoint lagao
3. Dynamic (x64dbg) → chala ke live values, decrypt data dekho
4. Wapas Static → nayi samajh ke saath code dubara padho
5. Patch (optional) → behaviour change karke test karo

Ye loop repeat hota rehta hai. Ek se clue milta hai, doosre se confirm hota hai.

4.5 — Anti-Debugging (Basic Idea)

Kuch programs (khaaskar malware) detect kar lete hain ki debugger laga hai, aur phir alag behave karte hain ya band ho jaate hain — taaki analysis mushkil ho.

Common anti-debug tricks: - `IsDebuggerPresent()` — Windows API jo batata hai debugger hai ya nahi. - Timing checks — code kitni der mein chala (debugger mein slow hota hai). - PEB flag check — process ke internal flag dekhna.

Bypass (basic): - `IsDebuggerPresent` ke return value ko patch karके 0 (no debugger) bana do. - Anti-debug check ko `nop` se hata do. - Plugins (jaise ScyllaHide) automatically kai anti-debug tricks chhupa dete hain.

Ye advanced topic hai. Abhi bas idea rakho ki aisa hota hai. Deep dive baad mein.

4.6 — Dynamic Example: Encrypted Password Nikaalna

Scenario: Program password ko reverse + XOR karke compare karta hai. Static mushkil.

Steps: 1. Ghidra mein dekha — comparison `memcmp` pe ho raha hai. 2. x64dbg mein program kholo. 3. `memcmp` pe breakpoint (`bp memcmp` ya address pe `F2`). 4. Program chalao, koi bhi password daalo. Breakpoint hit hoga. 5. `memcmp` ke arguments (registers/stack) dekho — ek argument mein tumhara input, doosre mein **asli expected password** (already decrypt hokar). 6. Memory dump mein wo asli password padho. Done! 🎯

Encryption ki logic samajhne ki zaroorat hi nahi padi — dynamic ne decrypt hui value seedha dikha di.

🧪 Phase 4 — Hands-On Practice

- Wahi crackmes (Phase 3 wale) ab **dynamic** analysis se solve karo.
- Ek crackme mein **patch** karke "wrong" ko "access granted" bana ke dekho.
- `strcmp` / `memcmp` pe breakpoint laga ke expected value pakdo.
- Static aur dynamic dono ka combo practice karo.

🎯 Key Takeaways (Ek Line Mein)

1. Dynamic analysis = program **chala ke**, rok-rok ke, live values dekhna.
2. **Breakpoint** sabse important — jahan chaho program ruk jaaye.
3. `F7` step into, `F8` step over, `Ctrl+F9` step out, `F9` run.

4. Comparison functions (`strcmp` / `memcmp`) pe breakpoint = asli password memory mein dikh jaata hai.
5. **Patching** = instructions badalna (`nop` , jump ulta) — behaviour change.
6. Static + dynamic ko **milaake** use karo — ek se clue, doosre se confirm.
7. **Anti-debugging** hoti hai — programs debugger detect karte hain; basic bypass patching/plugins se.
8. Unknown files hamesha **VM** mein.

✅ **Phase 4 Complete Jab:** Tum x64dbg/GDB mein breakpoint laga sako, live values/memory dekh sako, ek simple patch kar sako, aur ek crackme dynamic analysis se solve kar sako. Ab asli maidan — practice (Phase 5)! 🚀

Phase 4 — Interview Questions & Answers (Hinglish)

Dynamic Analysis se related important interview questions, simple Hinglish jawaab ke saath.

Q1. Dynamic analysis kya hai?

Ans: Dynamic analysis matlab program ko **actually chalakar**, beech-beech mein rok-rok ke, live values dekhkar samajhna. Static "book padhne" jaisa tha, dynamic "movie dekhne" jaisa hai — cheezein hoti hui dikhti hain, jaise register aur memory ki actual values.

Q2. Dynamic analysis kab zaroori ho jaata hai?

Ans: Jab code encrypted ya packed ho (static mein samajh na aaye), jab password/key runtime pe decrypt ya generate hota ho, ya jab logic itna complex ho ki chala ke dekhna aasan ho. Aisi situations mein static kaafi nahi hota.

Q3. Breakpoint kya hota hai?

Ans: Breakpoint ek nishaan hai jahan program apne aap ruk jaata hai, taaki hum us point pe registers, memory aur stack dekh sakein. Ye debugger ka sabse important tool hai. Types: software, conditional

(jab koi condition sach ho), aur hardware (memory read/write pe).

Q4. Step Into, Step Over aur Step Out mein kya farak hai?

Ans: - **Step Into (F7):** agar current line ek function call hai, uske andar ghus jao. - **Step Over (F8):** function ko poora chala do, but uske andar mat jao. - **Step Out (Ctrl+F9):** current function se bahar nikal jao (jahan se call hua tha).

Q5. Dynamic analysis ka sabse bada fayda kya hai?

Ans: Runtime pe **asli values** dikh jaati hain. Jaise agar password encrypted compare hota hai, static mein sirf encrypted bytes dikhenge, lekin comparison function pe breakpoint lagaakar hum decrypt hui asli value seedha memory mein dekh sakte hain — bina encryption samjhe.

Q6. Password nikaalne ke liye breakpoint kahan lagana chahiye?

Ans: Comparison functions pe — `strcmp`, `strncmp`, `memcmp`. Yahan program tumhare input ko expected (asli) value se compare karta hai, aur us waqt dono values (tumhara input + asli password) memory/registers mein dikh jaati hain. Input padhne wale functions (`scanf` / `fgets`) ke baad bhi useful hota hai.

Q7. Patching kya hoti hai?

Ans: Patching matlab program ke instructions/bytes ko badal ke uska behaviour change karna. Classic example: password check ke `jne` (jump if not equal) ko `je` ya `nop` mein badal dena — jisse galat password pe bhi "access granted" chalne lage.

Q8. NOP instruction kya karti hai aur patching mein kaise use hoti hai?

Ans: `nop` (No Operation, byte `0x90`) ka matlab "kuch mat karo, agli instruction pe jao". Patching mein hum kisi unwanted instruction (jaise ek check ya jump) ko `nop` se replace kar dete hain, taaki wo effectively hat jaaye aur program aage badh jaaye.

Q9. Static aur dynamic analysis ko saath mein kaise use karte hain?

Ans: Ek real workflow: pehle static (Ghidra) se overall structure aur interesting functions dhoondo, phir un functions pe dynamic (x64dbg) mein breakpoint lagaakar live values/decrypt data dekho, phir nayi

samajh ke saath wapas static code padho. Ye loop repeat hota hai — ek se clue milta hai, doosre se confirm hota hai.

Q10. Anti-debugging kya hota hai?

Ans: Ye techniques hain jinse program detect kar leta hai ki uspe debugger laga hai, aur phir alag behave karta hai ya band ho jaata hai — taaki analysis mushkil ho. Common tricks:

`IsDebuggerPresent()` API, timing checks (code slow chala toh debugger hai), aur PEB flags dekhna.

Q11. Anti-debugging ko bypass kaise karte hain (basic)?

Ans: `IsDebuggerPresent` jaise check ke return value ko patch karke 0 (no debugger) bana dete hain, ya anti-debug check ko `nop` se hata dete hain. Ghidra/x64dbg ke plugins (jaise ScyllaHide) automatically bahut si anti-debug tricks chhupa dete hain.

Q12. Malware ka dynamic analysis karte waqt kya precaution zaroori hai?

Ans: Malware ko hamesha ek isolated **Virtual Machine (VM)** mein chalao, kabhi apne main PC pe nahi. VM ka network isolate rakho aur snapshots use karo, taaki kuch bigad jaaye toh clean state pe wapas aa sako. Isse tumhara asli system aur data surakshit rehta hai.

Phase 4 — Interview Questions & Answers (English)

The same Dynamic Analysis interview questions, answered in clear English for formal interviews.

Q1. What is dynamic analysis?

Ans: Dynamic analysis means understanding a program by **actually running it**, pausing it, and observing live values. If static analysis is "reading a book," dynamic is "watching a movie" — you see things happen, such as the actual values in registers and memory.

Q2. When does dynamic analysis become necessary?

Ans: When the code is encrypted or packed (unclear statically), when a password/key is decrypted or generated at runtime, or when the logic is so complex that running it is easier than reading it. In these cases static analysis alone isn't enough.

Q3. What is a breakpoint?

Ans: A breakpoint is a marker where the program automatically pauses, so we can inspect registers, memory, and the stack at that point. It's the debugger's most important tool. Types include software, conditional (pause only when a condition is true), and hardware (pause on memory read/write).

Q4. What is the difference between Step Into, Step Over, and Step Out?

Ans: - **Step Into (F7):** if the current line is a function call, enter that function. - **Step Over (F8):** execute the function fully without entering it. - **Step Out (Ctrl+F9):** run until you exit the current function (back to its caller).

Q5. What is the biggest advantage of dynamic analysis?

Ans: You see the **actual values** at runtime. For example, if a password is compared in encrypted form, static analysis shows only encrypted bytes; but by placing a breakpoint at the comparison function, you can read the decrypted real value directly from memory — without understanding the encryption.

Q6. Where should you place a breakpoint to recover a password?

Ans: At comparison functions — `strcmp`, `strncmp`, `memcmp`. There the program compares your input against the expected value, and at that moment both values (your input and the real password) are visible in memory/registers. Right after input functions (`scanf` / `fgets`) is also useful.

Q7. What is patching?

Ans: Patching means changing a program's instructions/bytes to alter its behavior. A classic example: changing a password check's `jne` (jump if not equal) to `je` or `nop`, so that even a wrong password leads to "access granted."

Q8. What does the NOP instruction do and how is it used in patching?

Ans: `nop` (No Operation, byte `0x90`) means "do nothing, move to the next instruction." In patching we replace an unwanted instruction (like a check or a jump) with `nop`, effectively removing it so the program continues past it.

Q9. How do you use static and dynamic analysis together?

Ans: A real workflow: first use static (Ghidra) to find the overall structure and interesting functions, then set breakpoints on those functions in dynamic (x64dbg) to observe live values/decrypted data, then re-read the static code with new understanding. This loop repeats — one gives clues, the other confirms them.

Q10. What is anti-debugging?

Ans: These are techniques by which a program detects that a debugger is attached and then behaves differently or exits — to make analysis harder. Common tricks: the `IsDebuggerPresent()` API, timing checks (code runs slower under a debugger), and inspecting PEB flags.

Q11. How do you bypass anti-debugging (basics)?

Ans: Patch the return value of a check like `IsDebuggerPresent` to 0 (no debugger), or `nop` out the anti-debug check. Plugins for Ghidra/x64dbg (like ScyllaHide) automatically hide many anti-debug tricks.

Q12. What precautions are essential when dynamically analyzing malware?

Ans: Always run malware in an isolated **Virtual Machine (VM)**, never on your main PC. Keep the VM's network isolated and use snapshots, so you can revert to a clean state if something goes wrong. This keeps your real system and data safe.